

AI-Powered Code Documentation Generator

1. Problem Context

Every developer has opened a codebase and found functions with cryptic names, zero comments, and no explanation of *why* the code does what it does. This is especially painful for new hires, open-source contributors, or students joining a college project mid-way — onboarding can take days just to understand existing logic. Writing good documentation is tedious and low-priority under deadline pressure, so it's often skipped entirely, leading to "tribal knowledge" that lives only in one person's head. Existing tools either just extract function signatures (no real understanding) or require manual docstring writing — there's a gap for something that actually *reads and explains* code intelligently.

2. Objective

Build a web-based tool where a user can paste a code snippet, upload a file, or point to a small repo, and get auto-generated docstrings, a README, and plain-English explanations of functions — powered by an LLM — across multiple programming languages.

3. Core Features (Must-Have)

- **Code input methods:** paste code directly, upload a single file, or upload a small zipped folder/repo
- **Language detection:** auto-detect the programming language (Python, JavaScript, Java, C++ to start)
- **Docstring generation:** parse functions/classes and generate appropriate docstrings in the correct style (e.g., Google-style for Python, JSDoc for JavaScript)
- **"Explain this function" mode:** select any function and get a plain-English explanation of its purpose, inputs, outputs, and logic flow
- **README generator:** produce a basic README.md (project purpose, setup instructions, usage examples) based on the codebase structure
- **Output display + export:** show generated docs in-app with syntax highlighting, and let users download/copy the results
- **Basic error handling:** gracefully handle unsupported languages, broken/incomplete code, or files that are too large

4. Stretch Features (Good-to-Have)

- Inline diagram generation (e.g., simple flowchart of function logic) using Mermaid.js
- "Documentation quality score" that rates existing comments/docs and flags undocumented functions
- GitHub integration — paste a public repo URL and auto-generate docs for the whole repo
- Chat-style Q&A where users ask follow-up questions about the code ("why does this loop run twice?")

5. Suggested Tech Stack

- **Frontend:** React (with Vite for fast setup) — component-based, easy to build a code editor UI quickly. Use **Monaco Editor** (the engine behind VS Code) for a familiar code-input experience with syntax highlighting.
- **Backend:** Python with **FastAPI** — lightweight, fast to set up, and pairs naturally with AI/LLM API calls and code-parsing libraries.
- **AI/LLM:** Claude API or OpenAI API — send code as a prompt with clear instructions ("generate a docstring in Google style for this function") and parse the response. Start with one provider to save time.
- **Code parsing:** Use language-specific parsers where possible for structure detection — e.g., Python's built-in `ast` module, or **Tree-sitter** (supports multiple languages with one library) to identify functions/classes before sending them to the LLM. This makes results more reliable than just dumping raw text at the LLM.
- **Database:** Not strictly necessary for MVP — you can keep everything in-session/stateless. If you want to save history, use **SQLite** for simplicity.
- **Deployment:** Vercel (frontend) + Render or Railway (backend) — free tiers, fast to deploy for a demo.

6. Suggested Approach / Step-by-Step Plan

1. **Set up project skeleton** — React frontend + FastAPI backend, confirm they can talk to each other (simple "hello world" API call).
2. **Build the code input UI** — Monaco Editor for pasting/editing code, plus a file upload button.
3. **Integrate Tree-sitter or ast** to parse Python code first and extract function names, parameters, and bodies (get one language working end-to-end before adding more).
4. **Connect to the LLM API** — write a solid prompt template for docstring generation (test it in a playground first, e.g., "Given this function, write a concise docstring including params, return type, and purpose").
5. **Display generated docstrings** back in the UI, inserted next to/inside the original code, with copy/download options.
6. **Build "Explain this function" mode** — let users click/select a function and send it to the LLM with a different prompt template focused on plain-English explanation for a beginner.
7. **Build the README generator** — analyze overall file/folder structure and function list, then prompt the LLM to draft a README with sections (Overview, Setup, Usage).
8. **Add support for a second and third language** (JavaScript, Java) — reuse the same pipeline, just swap parsing logic and prompt templates.
9. **Polish UI/UX** — clean layout, loading states, error messages for bad input.
10. **Prepare demo** — pick 2-3 real-world messy code samples (from open-source repos) to show before/after transformation live.

7. Sample Data / APIs to Use

- **Real messy code samples:** pull functions from popular open-source GitHub repos (e.g., search GitHub for undocumented Python/JS utility files) — great for realistic demo material.
- **LLM API:** Claude API (console.anthropic.com) or OpenAI API — both offer free/trial credits suitable for a hackathon.
- **Parsing library:** Tree-sitter (open-source, supports 40+ languages) — no API key needed, runs locally.
- **Mock repo structure:** if GitHub integration feels risky, just prepare a small sample "project folder" (5-6 files) ahead of time to demo the README generator reliably without depending on live internet/API during judging.

8. Key Challenges to Anticipate

- **Inconsistent LLM output formatting** — the model might not always return clean docstrings. Tip: use structured prompts (ask for JSON output with fields like `function_name`, `docstring`, `explanation`) so parsing on your end is predictable.
- **Large codebases exceeding context limits** — sending an entire repo to the LLM at once can fail or get expensive. Tip: chunk by file/function and process one at a time, or limit MVP to single files/small repos.
- **Multi-language support complexity** — every language has different syntax and docstring conventions. Tip: get one language fully working and polished before expanding; don't split focus too early.
- **Prompt reliability across different code styles** — messy/incomplete code can confuse the LLM. Tip: test prompts against a range of sample inputs early, and add fallback messaging ("couldn't fully analyze this function") rather than crashing.

9. What a Good Demo Looks Like

Judges should see a live, real-time transformation: paste in a genuinely messy, undocumented function or file, and watch the tool generate a clean docstring, a plain-English explanation, and a polished README within seconds. The "explain this function" mode should feel like having a senior developer walk a newcomer through the code — clear, jargon-free, and accurate. Bonus points if the demo shows the same pipeline working across two different programming languages, proving the tool isn't hardcoded to one use case. Ending with a clear "before vs after" comparison (raw code vs. fully documented code) makes the value immediately obvious to non-technical judges too.